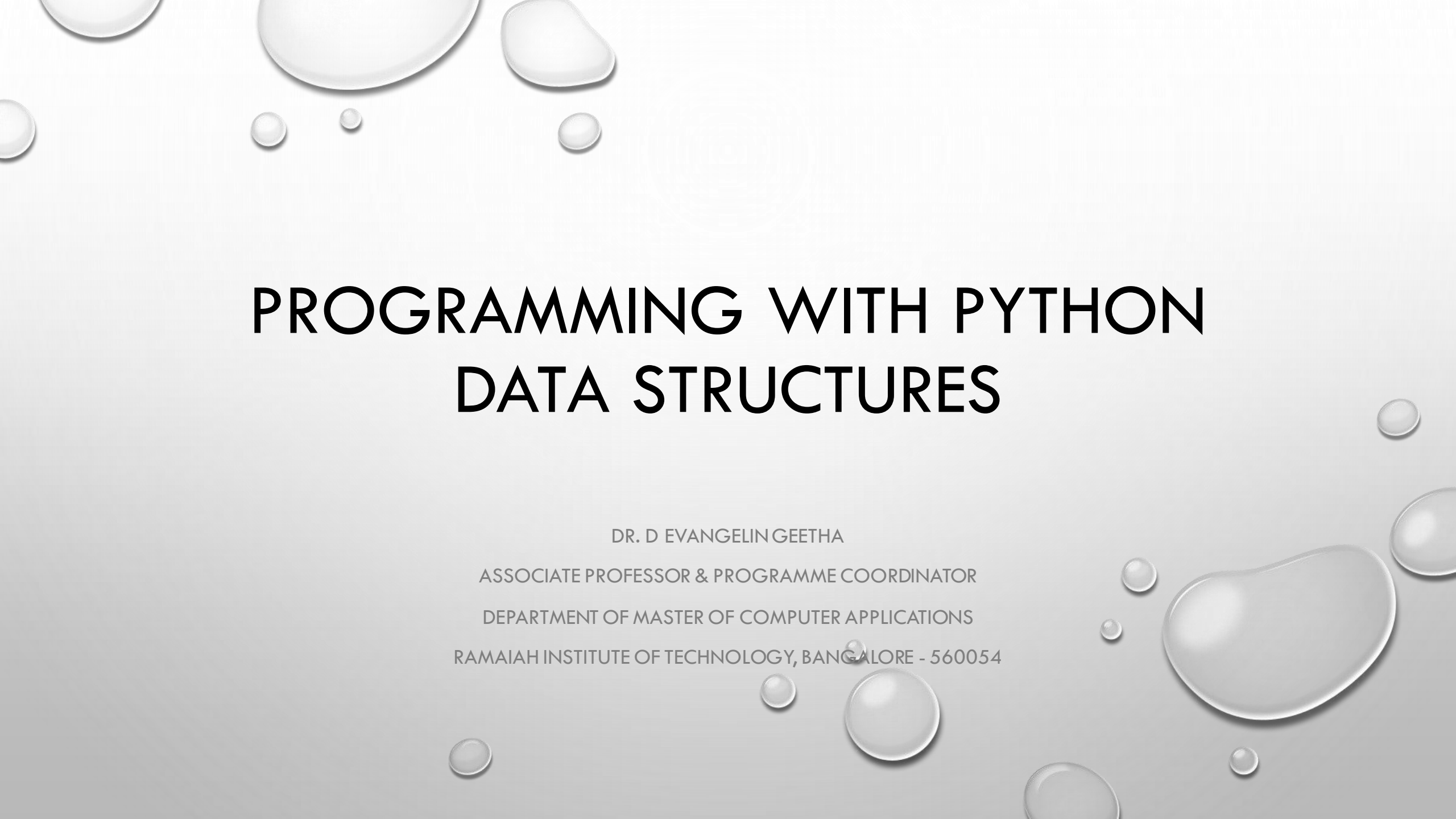


The background of the slide is a light gray gradient, decorated with numerous realistic water droplets of various sizes. Some droplets are large and prominent, while others are small and subtle. They are scattered across the slide, with a higher concentration in the top and bottom corners, creating a clean, modern, and fresh aesthetic.

PROGRAMMING WITH PYTHON DATA STRUCTURES

DR. D EVANGELINGEETHA

ASSOCIATE PROFESSOR & PROGRAMME COORDINATOR

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

RAMAIAH INSTITUTE OF TECHNOLOGY, BANGALORE - 560054

STRINGS

- **A compound data type**
- they're made up of smaller strings each containing one character.
- Strings in python are surrounded by either single quotation marks, or double quotation marks.
- 'hello' is the same as "hello".



STRINGS

- Assigning a string to a variable is done with the variable name followed by an equal sign and the string:
- `a = "Hello"`
`print(a)`



STRINGS

- Python does not have a character data type, a single character is simply a string with a length of 1.
- Square brackets can be used to access elements of the string.

- `a = "Hello, World!"`

```
print(a[1])
```

e



STRINGS

- The expression in brackets is called an index. An index specifies a member of an ordered collection, in this case the collection of characters in the string.
- The index indicates which one you want, hence the name. It can be any integer expression.



STRINGS

- `fruit = "banana"`
- `>>> letter = fruit[0]`
- `>>> print(letter)`
- `b`

STRINGS

- **String Length**
- To get the length of a string, use the `len()` function.
- The `len()` function returns the length of a string:
- `a = "Hello, World!"`
`print(len(a))`



STRINGS

- **String Length**
- To get the length of a string, use the `len()` function.
- The `len()` function returns the length of a string:
- `a = "Hello, World!"`
`print(len(a))`
- 13



STRINGS

```
word = "banana"
```

```
>>> len(word)
```

```
6
```

```
size = len(word)
```

```
last = word[size]
```



STRINGS

- `size = len(word)`
- `last = word[size-1]`
- we can use negative indices, which count backward from the end of the string.
- The expression `word[-1]` yields the last letter, `word[-2]` yields the second to last, and so on.

STRINGS

- **Looping Through a String**
- Since strings are arrays, we can loop through the characters in a string, with a for loop.
- ```
for x in "banana":
 print(x)
```

b  
a  
n  
a  
n  
a

# STRINGS

- Using while

```
fruit = 'Banana'
```

```
x = 0
```

```
while x < len(fruit):
```

```
 letter = fruit[ix]
```

```
 print(letter)
```

```
 x += 1
```



# STRINGS

- concatenation and a for loop to generate an abecedarian series.

Abecedarian refers to a series or list in which the elements appear in alphabetical order.

```
prefixes = "JKLMNOPQ"
```

```
suffix = "ack"
```

```
for p in prefixes:
```

```
 print(p + suffix)
```



# STRINGS

- **Output**

Jack

Kack

Lack

Mack

Nack

Oack

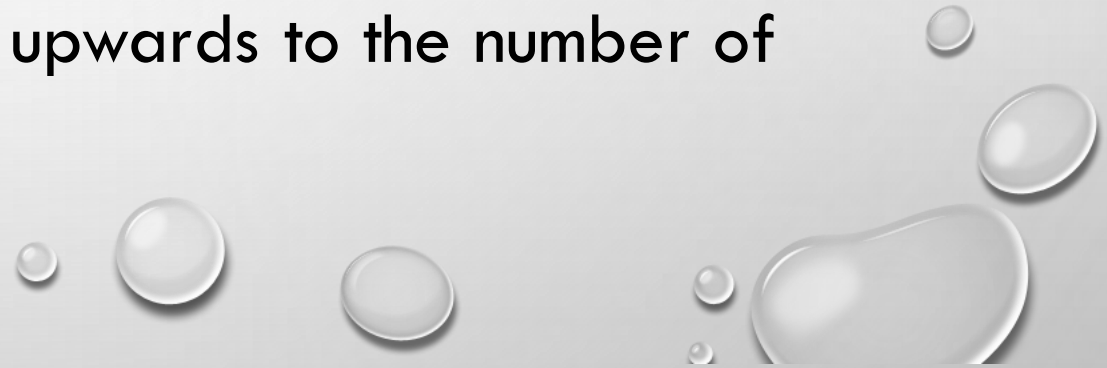
Pack

Qack



# INDEXING (SLICING)

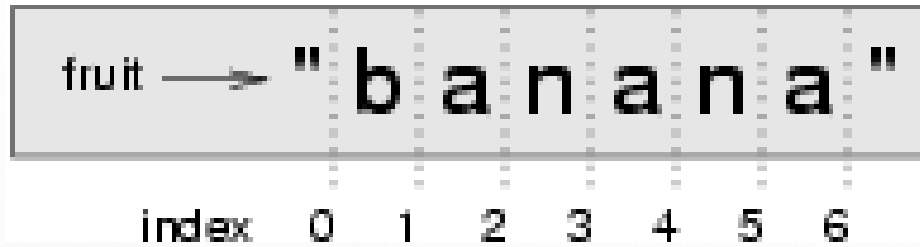
- Another common operation uses the **square brackets**. This is termed *indexing*, or *slicing*.
- An index or slice returns a portion of a larger value. In a string this can be used to produce a substring.
- Index values start at zero, and extend upwards to the number of characters in the string minus one.



# INDEXING (SLICING)

- When a single argument is given it is one character out of the string.
- When two integers are written, separated by a colon, it is termed a slice.
- A substring of a string is obtained by taking a slice. Similarly, we can slice a list to refer to some sublist of the items in the list
- The second value is an ending position. A portion of the string starting at the given position up to but not including the ending position is produced.





- `>>> word = "banana"`
- `>>> word[:3]`
- `'ban'`
- `>>> word[3:]`
- `'ana'`
- `>>> word[3:999]`
- `'ana'`

# INDEXING (SLICING)

```
>>> 'abc'[1]
```

```
'b'
```

```
>>> 'realtor'[2:6]
```

```
'alto'
```

If a colon is given but the last number is omitted the result is the remainder of the string starting from the given position.

```
>>> 'recurrent'[2:]
```

```
'current'
```

# INDEXING (SLICING)

If no number is given before the colon the result is the initial string up to but not including the given position

```
>>> 'halter'[:4]
```

```
'halt'
```



# NEGATIVE INDEXING

- Use negative indexes to start the slice from the end of the string:
- **Example**
- Get the characters:
- From: "o" in "World!" (position -5)
- To, but not included: "d" in "World!" (position -2):
- `b = "Hello, World!"`  
`print(b[-5:-2])`

or

# INDEXING (SLICING)

- Strings are *immutable*, meaning they cannot be changed once assigned.
- A new string can be constructed out of an existing using slicing commands.

```
>>> word = 'red'
```

```
>>> word[1] = 'a' # show word cannot be changed
```

# INDEXING (SLICING)

```
>>> word = 'b' + word[:2] + 'a' + word[2:]
```

```
>>> word # change red to bread
```

```
'bread'
```

# STRINGS-**STRING** COMPARISON

- The comparison operators work on strings. To see if two strings are equal:

```
word="banana"
```

```
if word == "banana":
```

```
 print("Yes, Equal!")
```



# STRINGS-**STRING** COMPARISON

- Other comparison operations are useful for putting words in lexicographical order:

**if** word < "banana":

    print("Your word, " + word + ", comes before banana.")

**elif** word > "banana":

    print("Your word, " + word + ", comes after banana.")

**else:**

    print("Yes, Equal!")



# STRINGS-**STRING** COMPARISON

- All the uppercase letters come before all the lowercase letters.
- Your word, Zebra, comes before banana.
- A common way to address this problem is to convert strings to a standard format, such as all lowercase, before performing the comparison.



# STRING OPERATIONS

| Method       | Description                                                                              |
|--------------|------------------------------------------------------------------------------------------|
| capitalize() | Converts the first character to upper case                                               |
| casefold()   | Converts string into lower case                                                          |
| count()      | Returns the number of times a specified value occurs in a string                         |
| find()       | Searches the string for a specified value and returns the position of where it was found |
| format()     | Formats specified values in a string                                                     |
| index()      | Searches the string for a specified value and returns the position of where it was found |
| isalnum()    | Returns True if all characters in the string are alphanumeric                            |
| isalpha()    | Returns True if all characters in the string are in the alphabet                         |

# STRING OPERATIONS

| Method      | Description                                                 |
|-------------|-------------------------------------------------------------|
| isdecimal() | Returns True if all characters in the string are decimals   |
| isdigit()   | Returns True if all characters in the string are digits     |
| islower()   | Returns True if all characters in the string are lower case |
| isnumeric() | Returns True if all characters in the string are numeric    |
| isupper()   | Returns True if all characters in the string are upper case |
| join()      | Converts the elements of an iterable into a string          |
| lower()     | Converts a string into lower case                           |

# STRING OPERATIONS

| Method       | Description                                                      |
|--------------|------------------------------------------------------------------|
| split()      | Splits the string at the specified separator, and returns a list |
| splitlines() | Splits the string at line breaks and returns a list              |
| startswith() | Returns true if the string starts with the specified value       |
| swapcase()   | Swaps cases, lower case becomes upper case and vice versa        |
| title()      | Converts the first character of each word to upper case          |
| upper()      | Converts a string into upper case                                |
| split()      | Splits the string at the specified separator, and returns a list |

# STRING OPERATIONS

- `capitalize()`-Converts the first character to upper case
- Example:

Upper case the first letter in this sentence:

```
txt = "hello, and welcome to my world."
```

```
x = txt.capitalize()
```

```
print (x)
```

Output:

Hello, and welcome to my world.



# STRING OPERATIONS

- `title()` - Converts the first character of each word to upper case
- Example

Make the first letter in each word upper case:

- ```
txt = "Welcome to my world"  
x = txt.title()  
print(x)
```

- Output:

Welcome To My World



STRING OPERATIONS

- `count()` - Returns the number of times a specified value occurs in a string
- Example

Where in the text is the word "welcome"?:

```
txt = "I love apples, apple are my favorite fruit"
```

```
x = txt.count("apple")  
print(x)
```

Output

2



STRING OPERATIONS

- `find()`-Searches the string for a specified value and returns the position of where it was found

Example:

Where in the text is the word "welcome"?:

```
txt = "Hello, welcome to my world."
```

```
x = txt.find("welcome")
```

```
print(x)
```

Output

7



STRING OPERATIONS

- `format()`-Formats specified values in a string

Example:

Insert the price inside the placeholder, the price should be in fixed point, two-decimal format:

```
txt = "For only {price:.2f} dollars!"  
print(txt.format(price = 49))
```

Output:

For only 49.00 dollars!



STRING OPERATIONS

- `index()` – Searches the string for a specified value and returns the position of where it was found
- Example:
- Where in the text is the word "welcome"?:

```
txt = "Hello, welcome to my world."  
x = txt.index("welcome")  
print(x)
```

Output:

7



STRING OPERATIONS

- `isalnum()`-Returns True if all characters in the string are alphanumeric

- Example

Check if all the characters in the text are alphanumeric

```
txt = "Company12"  
x = txt.isalnum()  
print(x)
```

- Output
True

- Example

```
txt = "Company@"  
x = txt.isalnum()  
print(x)
```

Output:
False

STRING OPERATIONS

- `isalpha()` - Returns True if all characters in the string are in the alphabet
- Example

Check if all the characters in the text are letters:

```
txt = "CompanyX"
```

```
x = txt.isalpha()
```

```
print(x)
```

- Output

True



STRING OPERATIONS

- `isdecimal()` - Returns True if all characters in the string are decimals
- Example:

Check if all the characters in a string are decimals (0-9):

```
txt = "1234"  
x = txt.isdecimal()  
print(x)
```

- Output
True



STRING OPERATIONS

- `isdigit()` - Returns True if all characters in the string are digits
- Example

Check if all the characters in the text are digits:

```
txt = "50800"  
x = txt.isdigit()  
print(x)
```

- Output
True



STRING OPERATIONS

- `islower()`-Returns True if all characters in the string are lower case

- Example

Check if all the characters in the text are in lower case:

```
txt = "hello world!"
```

```
x = txt.islower()
```

```
print(x)
```

- Output

True

```
txt = "Hello world!"
```

```
x = txt.islower()
```

```
print(x)
```

- Output

False

STRING OPERATIONS

- `isnumeric()`-Returns True if all characters in the string are numeric
- Example

Check if all the characters in the text are numeric:

```
txt = "565543"
```

```
x = txt.isnumeric()
```

```
print(x)
```

- Output

True



STRING OPERATIONS

- `isupper()`-Returns True if all characters in the string are upper case
- Example

Check if all the characters in the text are in upper case:

```
txt = "THIS IS NOW!"
```

```
x = txt.isupper()
```

```
print(x)
```

- Output

True



STRING OPERATIONS

- `join()`-Converts the elements of an iterable into a string
- Example

Join all items in a tuple into a string, using a hash character as separator:

```
myTuple = ("John", "Peter", "Vicky")
```

```
x = "#".join(myTuple)
```

```
print(x)
```

- Output

```
John#Peter#Vicky
```



STRING OPERATIONS

- `lower()`-Converts a string into lower case

- Example

```
txt = "Hello my FRIENDS"
```

```
x = txt.lower()
```

```
print(x)
```

- Output

```
hello my friends
```



STRING OPERATIONS

- `split()`-Splits the string at the specified separator, and returns a list

- Example

- Split a string into a list where each word is a list item:

```
txt = "welcome to the jungle"
```

```
x = txt.split()
```

```
print(x)
```

- Output

```
['welcome', 'to', 'the', 'jungle']
```



STRING OPERATIONS

- `splitlines()`-Splits the string at line breaks and returns a list
- Example
- Split a string into a list where each line is a list item:

```
txt = "Thank you for the music\nWelcome to the jungle"
```

```
x = txt.splitlines()
```

```
print(x)
```

- Output

```
['Thank you for the music', 'Welcome to the jungle']
```

STRING OPERATIONS

- `startswith()`-Returns true if the string starts with the specified value

- Example

- Check if the string starts with "Hello":

```
txt = "Hello, welcome to my world."
```

```
x = txt.startswith("Hello")
```

```
print(x)
```

- Output

True



STRING OPERATIONS

- `swapcase()`-Swaps cases, lower case becomes upper case and vice versa
- Example
- Make the lower case letters upper case and the upper case letters lower case:

```
txt = "Hello My Name Is PETER"  
x = txt.swapcase()  
print(x)
```

- Output

```
hELLO mY nAME iS peter
```



STRING OPERATIONS

- upper()-Converts the first character of each word to upper case
- Example
- Upper case the string:
- `txt = "Hello my friends"`
`x = txt.upper()`
`print(x)`
- Output

HELLO MY FRIENDS



LISTS

- We use lists in our everyday lives—we make **shopping lists, to-do lists, and mental checklists**.
- **Our DNA is essentially a long list of molecules** in the form of a double helix, found in the nucleus of all human cells and all living organisms.
- Lists are the most versatile of Python's compound data types.
- A list contains items separated by commas and enclosed within square brackets ([]).

STRINGS

- **Strings are immutable**
- It is tempting to use the `[]` operator on the left side of an assignment, with the intention of changing a character in a string.
- For example:
- `greeting = "Hello, world!"`
- `greeting[0] = 'J' # ERROR!`
- `print(greeting)`



STRINGS

- Strings are immutable, which means you can't change an existing string. The best you can do is create a new string that is a variation on the original:

```
greeting = "Hello, world!"
```

```
new_greeting = "J" + greeting[1:]
```

```
print(new_greeting)
```

- The solution here is to concatenate a new first letter onto a slice of greeting. This operation has no effect on the original string.

STRINGS

- **The in and not in operators**
- The in operator tests for membership. When both of the arguments to in are strings, in checks whether the left argument is a substring of the right argument.
- `>>> "p" in "apple"`
- True
- `>>> "i" in "apple"`
- False



STRINGS

- `>>> "ap" in "apple"`
- `True`
- `>>> "pa" in "apple"`
- `False`

STRINGS

- Note: A string is a substring of itself, and the empty string is a substring of any other string.
- `>>> "a" in "a"`
- True
- `>>> "apple" in "apple"`
- True
- `>>> "" in "a"`
- True
- `>>> "" in "apple"`
- True



STRINGS

- The not in operator returns the logical opposite results of in:
- `>>> "x" not in "apple"`
- `True`



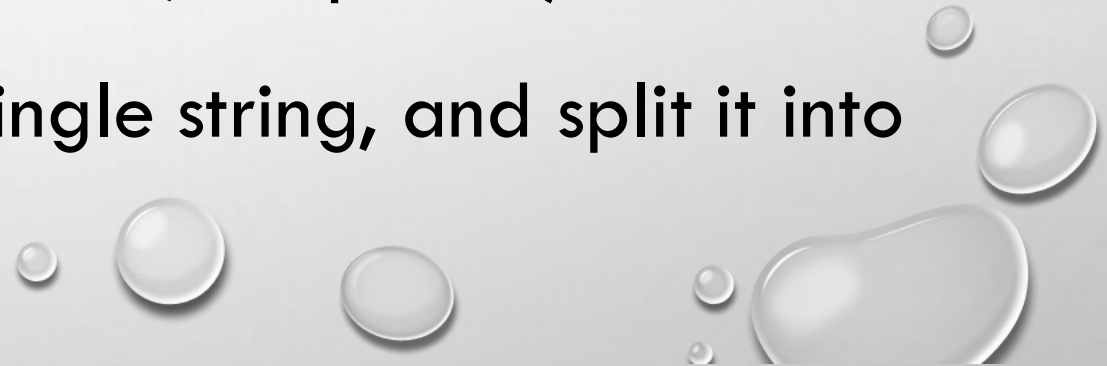
STRINGS

- **The built-in find method**
- It can find substrings in a string
- `>>> "banana".find("nan")`
- `2`
- `>>> "banana".find("na", 3)`
- `4`



STRINGS

- **The split method**
- it splits a single multi-word string into a list of individual words, removing all the whitespace between them.
(Whitespace means any tabs, newlines, or spaces.)
- This allows us to read input as a single string, and split it into words.



STRINGS

- **The split method**
- `phrase = "Well I never did said Alice"`
- `>>> words = phrase.split()`
- `>>> words`
- `['Well', 'I', 'never', 'did', 'said', 'Alice']`

STRINGS

- `>>> our_string = "Hello, World!"`
- `>>> all_caps = our_string.upper()`
- `>>> all_caps`
- `'HELLO, WORLD!'`
- `upper` is a method that can be invoked on any string object to create a new string, in which all the characters are in uppercase.

LISTS

- One of the differences between array and list is that all the items belonging to a list can be of different data type.
- The values stored in a list can be accessed using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the list and working their way to end -1.
- The plus (+) sign is the list concatenation operator, and the asterisk (*) is the repetition operator.

LIST CHARACTERISTICS

List Characteristics	Elements
Element Type	All elements of the same type
	Elements of different types
Length	Fixed length
	Varying length
Modifiability	Mutable (alterable)
	Immutable (unalterable)
Common Operations	Determine if a list is empty
	Determine the length of a list
	Access (retrieve) elements of a list
	Insert elements into a list
	Replace elements of a list
	Delete elements of a list
	Append elements to (the end of) a list

LISTS

- A list is a **linear data structure**, meaning that its elements have a linear ordering.
- That is, there is a first element, a second element, and so on.
- The **values** that make up a list are called its **elements**, or its items

0:	68.8
1:	70.2
2:	67.2
3:	71.8
4:	73.2
5:	75.6
6:	74.0

LISTS

- A list is created using square brackets.
- At its simplest, a list is simply a collection of values, separated by commas:

```
>>>lst = [1,4,7,9,12] # Creates a list with 5 elements
```

```
>>>lst
```

```
[1, 4, 7, 9, 12]
```



LISTS

- There is no requirement that a list contain elements of the same type.
- The following creates a list consisting of an integer, a real, a string, and another list.

```
>>>lst2 = [1,2.4,'abc',lst]
```

```
>>>lst2
```

```
[1, 2.4, 'abc', [1, 4, 7, 9, 12]]
```

- An open bracket can start a list that extends over multiple lines:

```
lst3 = [1,2,
```

```
        3,4,
```

```
        5]
```

```
print(lst3)
```

Output:

```
[1, 2, 3, 4, 5]
```


ACCESSING VALUES IN A LISTS

```
list1 = ['physics', 'chemistry', 1997, 2000]
```

```
list2 = [1, 2, 3, 4, 5, 6, 7]
```

```
print ("list1[0]: ", list1[0])
```

```
print (list2[1:5]: ", list1[1:5])
```

- Output

```
list1[0]: physics
```

```
list2[1:5]: [2, 3, 4, 5]
```



LISTS

- `list = [ 'abcd', 786 , 2.23, 'john', 70.2 ]`
- `tinylist = [123, 'john']`
- `print (list)` `# Prints complete list`
- `print (list[0])` `# Prints first element of the list`
- `print (list[1:3])` `# Prints elements starting from 2nd till 3rd`
- `print (list[2:])` `# Prints elements starting from 3rd element`
- `print (tinylist * 2)` `# Prints list two times`
- `print (list + tinylist)` `# Prints concatenated lists`

LISTS

- `list = [ 'abcd', 786 , 2.23, 'john', 70.2 ]`
- `tinylist = [123, 'john']`
- `print (list)` `# Prints complete list`
- `print (list[0])` `# Prints first element of the list`
- `print (list[1:3])` `# Prints elements starting from 2nd till 3rd`
- `print (list[2:])` `# Prints elements starting from 3rd element`
- `print (tinylist * 2)` `# Prints list two times`
- `print (list + tinylist)` `# Prints concatenated lists`

```
list = [ 'abcd', 786 , 2.23, 'john', 70.2 ]
```

```
tinylist = [123, 'john']
```

- Output

```
['abcd', 786, 2.23, 'john',  
70.20000000000000003]
```

```
abcd
```

```
[786, 2.23]
```

```
[2.23, 'john', 70.20000000000000003]
```

```
[123, 'john', 123, 'john']
```

```
['abcd', 786, 2.23, 'john',  
70.20000000000000003, 123, 'john']
```

```
lst = [1,4,7,9,12]
>>>print("The first Element is:
",lst[0])
The first Element is:    1
>>>lst[2]
7
>>>lst[1:3]
[4, 7]
>>>lst[2:]
[7, 9, 12]
>>>lst[:2]
[1, 4]
```

```
>>>4 in lst
True
>>>5 in lst
False
>>>lst+[3,4]
[1, 4, 7, 9, 12, 3, 4]
>>>len(lst)
5
```

```
>>> [2,3]*4
[2, 3, 2, 3, 2, 3, 2, 3]
>>>max(lst)
12
>>>min(lst)
1
>>>list('abc')
['a', 'b', 'c']
>>>max(lst2)
>>len(lst2) # Counts list as a
single element
4
```

UPDATING LISTS

- A list is *mutable*.
- The elements in a list can be changed after it is created.

1. It can happen as a result of using an index as target of an assignment:

```
lst = [1,4,7,9,12]
```

```
>>>lst[2]=5
```

```
>>>lst
```

```
[1, 4, 5, 9, 12]
```

UPDATING LISTS

2. A **slice** can also be the target of an assignment. The value on the right hand side must also be a list, however it is not necessary that this list have the same number of elements as the slice.

The list will be enlarged or reduced in size as necessary. (Delete and replace)

```
[1, 4, 5, 9, 12]
```

```
>>>lst[1:3]=[9,8,7,6]
```

```
>>>lst
```

```
[1, 9, 8, 7, 6, 9, 12]
```



UPDATING LISTS

- The del statement can be used to delete an element from a list.

```
>>>del lst[3]
```

```
>>>lst
```

```
[1, 9, 8, 6, 9, 12]
```



LIST OPERATIONS

- Lists provide a number of operations written using dot notation.
- The list being modified is followed by a dot, then the name of the operation and an argument list.
- `s.append(x)` - Appends element `x` to `s`
- `s.extend(ls)` - Appends (extends) list `s` with list `ls`
- `s.count(x)` - Count number of occurrences of `x` in `s`
- `s.index(x)` - Returns the index of first occurrence of `x`

LIST OPERATIONS

- `s.pop()` - Returns and removes last element from `s`
- `s.pop(i)` - Return and removes element `i` from `s`
- `s.remove(x)` - Searches for `x` and removes it from `s`
- `s.reverse()` - Reverse elements of `s` in place
- `s.sort()` - Sort elements of `s` into ascending order
- `s.insert(i, x)` - Inserts `x` at location `i`



LIST OPERATIONS

```
>>>lst=[1,7,3,9,2]
```

```
>>>lst
```

```
[1, 7, 3, 9, 2]
```

```
>>>lst.append(4)
```

```
>>>lst
```

```
[1, 7, 3, 9, 2, 4]
```

```
>>>lst.extend([8,5])
```

```
>>>lst
```

```
[1, 7, 3, 9, 2, 4, 8, 5]
```

```
>>>lst.count(5)
```

```
1
```

```
>>>lst.index(5)
```

```
7
```

```
>>>lst.extend([9,3])
```

```
>>>lst
```

```
[1, 7, 3, 9, 2, 4, 8, 5, 9, 3]
```

```
>>>lst.count(9)
```

```
2
```

```
>>>lst.pop()
```

```
3
```

```
>>>lst
```

```
[1, 7, 3, 9, 2, 4, 8, 5, 9]
```

```
>>>lst.sort()
```

```
>>>lst
```

```
[1, 2, 3, 4, 5, 7, 8, 9, 9]
```

```
>>>lst.pop(3)
```

```
4
```

```
>>>lst
```

```
[1, 2, 3, 5, 7, 8, 9, 9]
```

```
>>>lst.remove(9)
```

If the list contains more than one matching the specified value, only the first is deleted.

LIST OPERATIONS

```
>>>lst
```

```
[1, 2, 3, 5, 7, 8, 9]
```

```
>>>lst.reverse()
```

```
>>>lst
```

```
[9, 8, 7, 5, 3, 2, 1]
```

```
>>>lst.insert(3,6)
```

```
>>>lst
```

```
[9, 8, 7, 6, 5, 3, 2, 1]
```

```
>>>lst2
```

```
[1, 2.4, 'abc', [1, 9, 8, 6, 9,  
12]]
```

```
>>>lst2.count(1)      #
```

Match with elements only

```
1
```

```
>>>lst2.count ([1, 9, 8, 6,  
9, 12]) # Match with list
```

```
1
```

LIST OPERATIONS - APPEND VS EXTEND

- **“extend”** takes a single argument, which is always a list, and adds each of the elements of that list to the original list.
- **”append”** takes one argument, which can be any data type, and simply **adds it to the end of the list**



LIST OPERATIONS - APPEND VS EXTEND

- **nums.append([6,7,8])**

```
Initial list is  
[1, 2, 3, 4, 5]  
After appending  
[1, 2, 3, 4, 5, [6, 7, 8]]
```

- **nums.extend([6,7,8])**

```
Initial list is  
[1, 2, 3, 4, 5]  
After extending  
[1, 2, 3, 4, 5, 6, 7, 8]
```

LISTS AND LOOPS

```
>>> lst = [3, 5, 7]
```

```
>>> for x in lst: print x
```

3

5

7



ZIP() METHOD

- **Python zip() method** takes iterable or containers and returns a single iterator object, having mapped values from all the containers.
 - It is used to map the similar index of multiple containers so that they can be used just using a single entity.
 - **Syntax :** *zip(*iterators)*
 - **Parameters :** *Python iterables or containers (list, string etc)*
- Return Value :** *Returns a single iterator object, having mapped values from all the containers*

ZIP() METHOD

- An interesting function named zip takes two lists, and produces a list of tuples representing their pairs:
- `>>> zip([1, 2, 3],[5, 6, 7])`
- `[(1, 5), (2, 6), (3, 7)]`



ZIP() METHOD

- When combined with the multiple assignment you learned about in chapter 2, this produces an easy and elegant way to loop over two lists in parallel:

```
>>> one = [1, 2, 3]
```

```
>>> two = [5, 6, 7]
```

```
>>> for x, y in zip(one, two):
```

```
... print 'x', x,'and y',y
```

```
x 1 and y 5
```

```
x 2 and y 6
```

```
x 3 and y 7
```



ZIP() METHOD

```
name = [ "Manjeet", "Nikhil", "Shambhavi", "Astha" ]
```

```
roll_no = [ 4, 1, 3, 2 ]
```

```
# using zip() to map values
```

```
mapped = zip(name, roll_no)
```

```
print (mapped)
```

```
print(set(mapped))
```

Output:

```
{('Shambhavi', 3), ('Nikhil', 1), ('Astha', 2), ('Manjeet', 4)}
```

HOW TO UNZIP?

- Unzipping means converting the zipped values back to the individual self as they were. This is done with the help of “*” operator.



UNZIPPING

Python code to demonstrate the working of unzip

initializing lists

```
name = ["Manjeet", "Nikhil", "Shambhavi", "Astha"]
```

```
roll_no = [4, 1, 3, 2]
```

```
marks = [40, 50, 60, 70]
```

using zip() to map values

```
mapped = zip(name, roll_no, marks)
```



UNZIPPING

```
# converting values to print as list
mapped = list(mapped)

# printing resultant values
print("The zipped result is : ", end="")
print(mapped)
print("\n")

#unzipping values
namz, roll_noz, marksz = zip(*mapped)
print("The unzipped result: \n", end="")
```



UNZIPPING

printing initial lists

```
print("The name list is : ", end="")
```

```
print(namz)
```

```
print("The roll_no list is : ", end="")
```

```
print(roll_noz)
```

```
print("The marks list is : ", end="")
```

```
print(marksz)
```



ASSIGNMENT AND REFERENCES

- assignment copies references and not values
- create a list and assign it to a variable named **a**.
- assign the variable **a** to another variable **b**.
- make a change to part of b.
- What do you think will be the value being held in a?



ASSIGNMENT AND REFERENCES

```
>>> a = [1, 2, 3]
```

```
>>> b = a
```

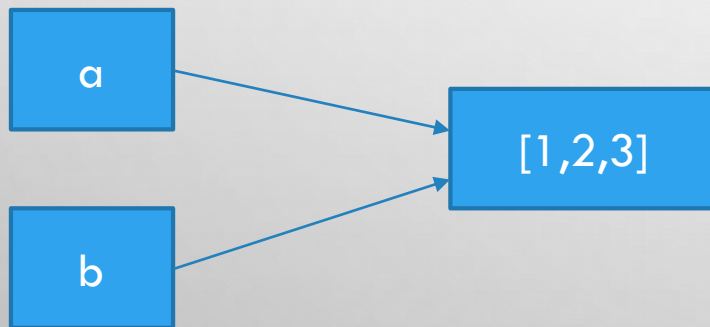
```
>>> b[1] = 7
```

```
>>> a
```

```
[1, 7, 3]
```


ASSIGNMENT AND REFERENCES

- assignment makes the name on the left *refer* to the same object as the value on the right.
- the name **b** is an *alias* for the value stored in variable **a**.
- In technical terms, we say that Python uses *reference assignment*.



ASSIGNMENT AND REFERENCES

- to avoid the trap of reference assignment by making a true copy, use a slice

```
>>> a = [1, 2, 3]
```

```
>>> b = a[:] # use a slice instead of a simple name
```

```
>>> b[1] = 7
```

```
>>> a
```

```
[1, 2, 3]
```

- The slice makes a copy of the value. Now a change to one will not alter the other.

IDENTITY AND EQUALITY

- The operator, `==`, is testing equality
- The identity testing operator is called `is`
- Two lists are equal if they are the same length and their corresponding elements are equal
- Two lists are identical if they are exactly the same object.



IDENTITY AND EQUALITY

```
if [1, 2, 3] == [1, 2, 3]: print 'equal'
```

```
equal
```

```
>>> if [1, 2, 3] is [1, 2, 3]: print 'identical'
```

```
>>>
```

```
>>> a = [1, 2, 3]
```

```
>>> b = a
```

```
>>> if a is b: print 'identical'
```

```
identical
```



IDENTITY AND EQUALITY

```
a = [1, 2, 3]
```

```
b = a
```

```
if a is b: print ('identical')
```

```
if [1, 2, 3] is [1, 2, 3]:
```

```
    print('identical')
```

```
else:
```

```
    print('Not identical')
```

```
a = [1,2,3]
```

```
b = [1,2,3]
```

```
print ('identical') if a is b else print ('Not identical')
```



IDENTITY AND EQUALITY

```
a = [1, 2, 3]
b = a
if a is b: print ('identical')
if [1, 2, 3] is [1, 2, 3]:
    print('identical')
else:
    print('Not identical')
a = [1,2,3]
b = [1,2,3]
print ('identical') if a is b else print ('Not identical')
```

Output:

identical

Not identical

Not identical



TUPLES

- A *tuple* is similar to a list, but is formed using parenthesis rather than square brackets.
- Like strings, tuples are immutable, meaning they cannot be changed once they are created.
- In all other respects they are identical to a list. This means that any of the list operations that do not change the value of the tuple are valid.
- A list can be changed into a tuple, and vice versa.

TUPLES

- Creating a tuple is as simple as putting different comma-separated values.
- Optionally, you can put these comma-separated values between parentheses

```
also tup1 = ('physics', 'chemistry',  
            1997, 2000)
```

```
tup2 = (1, 2, 3, 4, 5 )
```

```
tup3 = "a", "b", "c", "d"
```

```
print("tuple 1", tup1)
```

```
print("tuple 2", tup2)
```

```
print("tuple 3", tup3)
```

```
print(type(tup3))
```

Output:

```
tuple 1 ('physics', 'chemistry',  
1997, 2000)
```

```
tuple 2 (1, 2, 3, 4, 5)
```

```
tuple 3 ('a', 'b', 'c', 'd')
```

```
<class 'tuple'>
```


TUPLES

- The empty tuple is written as two parentheses containing nothing –

```
tup1 = ();
```

- To write a tuple containing a single value you have to include a comma, even though there is only one value –

```
tup1 = (50,)
```

- Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

TUPLES

```
tup1 = (50)
```

```
print("tuple 1", tup1)
```

```
print("tup1[0]", tup1[0])
```

```
tup1 50
```

Traceback (most recent call last):

File "E:/Python_2021/Python 2022/tup_example 2.py", line 3, in
<module>

```
    print("tup1[0]", tup1[0])
```

TypeError: 'int' object is not subscriptable

TUPLES

```
tup1 = (50,)
print("tuple 1", tup1)
print("tup1[0]", tup1[0])
```

```
tuple 1 (50,)
tup1[0] 50
```



ACCESSING VALUES IN TUPLES

- To access values in tuple, use the square brackets for slicing along with the index or indices to obtain the value available at that index

```
tup1 = ('physics', 'chemistry', 1997, 2000)
```

```
tup2 = (1, 2, 3, 4, 5, 6, 7 )
```

```
print ("tup1[0]: ", tup1[0])
```

```
print ("tup2[1:5]: ", tup2[1:5])
```



ACCESSING VALUES IN TUPLES

When the above code is executed, it produces the following result –

```
tup1[0]: physics
```

```
tup2[1:5]: (2, 3, 4, 5)
```



UPDATING TUPLES

- Tuples are immutable, which means values of tuple elements cannot be updated or changed
- You are able to take portions of the existing tuples to create new tuples

```
tup1 = (12, 34.56)
```

```
tup2 = ('abc', 'xyz')
```



UPDATING TUPLES

Following action is not valid for tuples

```
tup1[0] = 100;
```

Traceback (most recent call last):

File "E:/Python_2021/Python 2022/tup_example 3.py", line 5, in <module>

```
tup1[0] = 100;
```

TypeError: 'tuple' object does not support item assignment

So let's create a new tuple as follows

```
tup3 = tup1 + tup2
```

```
print (tup3)
```

UPDATING TUPLES

```
tup1 = (12, 34.56)
```

```
tup2 = ('abc', 'xyz')
```

So let's create a new tuple as follows

```
tup3 = tup1 + tup2
```

```
print (tup3)
```

Output:

```
(12, 34.56, 'abc', 'xyz')
```



UPDATING TUPLES

- `zoo = ('python', 'tiger', 'elephant');`
- `print (zoo)`
- `zoo_new = ('zeebra', 'lion', zoo)`
- `print (zoo_new)`
- `print("animals brought from old zoo are:", zoo_new[2])`
- `print("animal 2 from old zoo is", zoo_new[2][1])`
- `print("Number of cages required: ", len(zoo_new)-1+len(zoo_new[2]))`

DELETE TUPLE ELEMENTS

- Removing individual tuple elements is not possible.
- putting together another tuple with the undesired elements discarded
- To explicitly remove an entire tuple, just use the **del** statement.



DELETE TUPLE ELEMENTS

```
tup = ('physics', 'chemistry', 1997, 2000);
```

```
print (tup)
```

```
del tup[2]
```

Output:

```
('physics', 'chemistry', 1997, 2000)
```

Traceback (most recent call last):

File "E:/Python_2021/Python 2022/tup_example 4.py", line 4, in <module>

```
del tup[2]
```

TypeError: 'tuple' object doesn't support item deletion

DELETE TUPLE ELEMENTS

```
tup = ('physics', 'chemistry', 1997, 2000);
```

```
print (tup)
```

```
del tup;
```

```
print ("After deleting tup : ")
```

```
print (tup)
```

Output:

('physics', 'chemistry', 1997, 2000)

After deleting tup :

Traceback (most recent call last):

File "E:/Python_2021/Python
2022/tup_example 4.py", line 7, in
<module>

print (tup)

NameError: name 'tup' is not defined

BASIC TUPLES OPERATIONS

Python Expression	Results	Description
<code>len((1, 2, 3))</code>	3	Length
<code>(1, 2, 3) + (4, 5, 6)</code>	(1, 2, 3, 4, 5, 6)	Concatenation
<code>('Hi!',) * 4</code>	('Hi!', 'Hi!', 'Hi!', 'Hi!')	Repetition
<code>3 in (1, 2, 3)</code>	True	Membership
<code>for x in (1,2,3) : print (x, end = ' ')</code>	1 2 3	Iteration

BASIC TUPLES OPERATIONS

- Indexing, Slicing, and Matrixes
- T=('C++', 'Java', 'Python')

Python Expression	Results	Description
T[2]	'Python'	Offsets start at zero
T[-2]	'Java'	Negative: count from the right
T[1:]	('Java', 'Python')	Slicing fetches sections

BUILT-IN TUPLE FUNCTIONS

Sr.No.	Function & Description
1	<u>len(tuple)</u> Gives the total length of the tuple.
2	<u>max(tuple)</u> Returns item from the tuple with max value.
3	<u>min(tuple)</u> Returns item from the tuple with min value.
4	<u>tuple(seq)</u> Converts a list into tuple.

LEN() METHOD

- The len() method returns the number of elements in the tuple.
- Syntax

Following is the syntax for len() method –

len(tuple)

Parameters

- tuple – This is a tuple for which number of elements to be counted.
- Return Value

This method returns the number of elements in the tuple.

LEN() METHOD

- Example

example shows the usage of len() method

```
tuple1, tuple2 = (1 23, 'xyz', 'zara'), (456, 'abc')
```

```
print ("First tuple length : ", len(tuple1))
```

```
print ("Second tuple length : ", len(tuple2))
```

- Result

When we run above program, it produces the following result –

First tuple length : 3

Second tuple length : 2

MAX() METHOD

- The max() method returns the elements from the tuple with maximum value.

- Syntax

max(tuple)

- Parameters

tuple – This is a tuple from which max valued element to be returned.

- Return Value

This method returns the elements from the tuple with maximum value.

MAX() METHOD

- Example

```
tuple1, tuple2 = ('maths', 'che', 'phy', 'bio'), (456, 700, 200)
```

```
print ("Max value element : ", max(tuple1))
```

```
print ("Max value element : ", max(tuple2))
```

- Result

Max value element : phy

Max value element : 700

MIN() METHOD

- The min() method returns the elements from the tuple with minimum value.

- Syntax

`min(tuple)`

- Parameters

tuple – This is a tuple from which min valued element to be returned.

- Return Value

This method returns the elements from the tuple with minimum value.

MIN() METHOD

- Example

```
tuple1, tuple2 = ('maths', 'che', 'phy', 'bio'), (456, 700, 200)
```

```
print ("min value element : ", min(tuple1))
```

```
print ("min value element : ", min(tuple2))
```

- Result

```
min value element : bio
```

```
min value element : 200
```

TUPLE() METHOD

- The tuple() method converts a list of items into tuples.

- Syntax

`tuple( seq )`

- Parameters

`seq` – This is a tuple to be converted into tuple.

- Return Value

This method returns the tuple.



TUPLE() METHOD

- Example

```
list1 = ['maths', 'che', 'phy', 'bio']
```

```
tuple1 = tuple(list1)
```

```
print ("tuple elements : ", tuple1)
```

- Result

```
tuple elements : ('maths', 'che', 'phy', 'bio')
```

TUPLE() METHOD

```
list1 = ['maths', 'che', 'phy', 'bio']
```

```
print("List Elements: ", list1)
```

```
list1[2] = 'physics'
```

```
tuple1 = tuple(list1)
```

```
print ("tuple elements : ", tuple1)
```

```
tuple1[2] = 'phy'
```



TUPLE() METHOD

- Output

List Elements: ['maths', 'che', 'phy', 'bio']

tuple elements : ('maths', 'che', 'physics', 'bio')

Traceback (most recent call last):

File "E:/Python_2021/Python 2022/tup_example 5.py", line 7, in
<module>

```
tuple1[2] = 'phy'
```

TypeError: 'tuple' object does not support item assignment

DICTIONARY

- Dictionaries are used to store data values in key:value pairs.
- A dictionary is a collection which is changeable and do not allow duplicates.
- Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces.
- An empty dictionary without any items is written with just two curly braces, like this: {}.
- Keys are unique within a dictionary while values may not be.
- The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

DICTIONARY

```
# empty dictionary
```

```
my_dict = {}
```

```
# dictionary with integer keys
```

```
my_dict = {1: 'apple', 2: 'ball'}
```

```
print("dictionary with integer keys: ", my_dict)
```

```
# dictionary with mixed keys
```

```
my_dict = {'name': 'John', 1: [2, 4, 3]}
```

```
print("dictionary with mixed keys: ", my_dict)
```

DICTIONARY

```
# using dict()
```

```
my_dict = dict({1:'apple', 2:'ball'})
```

```
print("dictionary using dict(): ", my_dict)
```

```
# from sequence having each item as a pair
```

```
my_dict = dict([(1,'apple'), (2,'ball')])
```

```
print("from sequence having each item as a pair: ", my_dict)
```

ACCESSING VALUES IN DICTIONARY

- To access dictionary elements, use the square brackets along with the key to obtain its value.

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
```

```
print ("Name: ", dict1['Name'])
```

```
print ("Age: ", dict1['Age'])
```

Output:

Name: Zara

Age: 7



ACCESSING VALUES IN DICTIONARY

- If we attempt to access a data item with a key, which is not a part of the dictionary, we get an error as follows:

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
print ("dict1['Alice']:", dict1['Alice'])
```

- Output

Traceback (most recent call last):

File "E:\Python_2021\Python 2022\Dict_Ex3.py", line 5, in <module>

```
print ("dict1['Alice']:", dict1['Alice'])
```

KeyError: 'Alice'

UPDATING DICTIONARY

- update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown in a simple example given below.

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
```

```
dict1['Age'] = 8; # update existing entry
```

```
dict1['School'] = "DPS School" # Add new entry
```

```
print ("dict1['Age']: ", dict1['Age'])
```

```
print ("dict1['School']: ", dict1['School'])
```

```
print("Dictionary is: ", dict1)
```

- Output

dict1['Age']: 8

dict1['School']: DPS School

Dictionary is: {'Name': 'Zara', 'Age': 8, 'Class': 'First', 'School': 'DPS School'}

DELETE DICTIONARY ELEMENTS

- You can either remove individual dictionary elements or clear the entire contents of a dictionary.
- You can also delete entire dictionary in a single operation.
- To explicitly remove an entire dictionary, just use the del statement.



DELETE DICTIONARY ELEMENTS

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
print("Type: ",type(dict1))  
print(dict1)  
del dict1['Name'] # remove entry with key 'Name'  
print(dict1)  
dict1.clear()    # remove all entries in dict  
print(dict1)  
del dict1        # delete entire dictionary  
print(dict1)  
print ("dict1['Age']:", dict1['Age'])  
print ("dict1['School']:", dict1['School'])
```



DELETE DICTIONARY ELEMENTS

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
print("Type: ", type(dict1))  
print(dict1)  
del dict1['Name'] # remove entry with key 'Name'  
print(dict1)  
dict1.clear()    # remove all entries in dict  
print(dict1)  
del dict1        # delete entire dictionary  
print(dict1)  
print ("dict1['Age']:", dict1['Age'])  
print ("dict1['School']:", dict1['School'])
```

```
Type: <class 'dict'>  
{'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
{'Age': 7, 'Class': 'First'}
```

```
{}
```

Traceback (most recent call last):

File "E:/Python_2021/Python
2023/dict3.py", line 9, in <module>

```
print(dict1)
```

NameError: name 'dict1' is not defined.
Did you mean: 'dict'?

PROPERTIES OF DICTIONARY KEYS

- Dictionary values have no restrictions.
- They can be any arbitrary Python object, either standard objects or user-defined objects.
- However, same is not true for the keys.



PROPERTIES OF DICTIONARY KEYS

- There are two important points to remember about dictionary keys
 - More than one entry per key is not allowed.
 - This means no duplicate key is allowed.
 - When duplicate keys are encountered during assignment, the last assignment wins.

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Name': 'Mani'}  
print ("dict1['Name']:", dict1['Name'])
```

- Output:

`dict1['Name']:` Mani

PROPERTIES OF DICTIONARY KEYS

- Keys must be immutable.
- This means you can use strings, numbers or tuples as dictionary keys but something like ['key'] is not allowed.

```
dict1 = {('Name','USN'): ['Zara','1MS13MCA45'], 'Age':9}
```

```
print(dict1)
```

```
print("Name: ", dict1[('Name','USN')])
```

```
dict1[('Name','USN')] = ['Angel','1MS13MCA50']
```

```
print("Name: ", dict1[('Name','USN')])
```

```
dict2 = {['Name']: 'Zara', 'Age': 7}
```

```
print ("dict2['Name']: ", dict2['Name'])
```

```
{('Name', 'USN'): ['Zara', '1MS13MCA45'], 'Age': 9}
```

```
Name: ['Zara', '1MS13MCA45']
```

```
Name: ['Angel', '1MS13MCA50']
```

```
Traceback (most recent call last):
```

```
File "E:/Python_2021/Python  
2022/Dict_Ex7.py", line 10, in <module>
```

```
dict2 = {['Name']: 'Zara', 'Age': 7}
```

```
TypeError: unhashable type: 'list'
```

BUILT-IN DICTIONARY FUNCTIONS AND METHODS

Sr.No	Function & Description
1	<u>len(dict)</u> Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.
2	<u>str(dict)</u> Produces a printable string representation of a dictionary
3	<u>type(variable)</u> Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

BUILT-IN DICTIONARY FUNCTIONS AND METHODS

Sr.No	Method & Description
1	<u>dict.clear()</u> Removes all elements of dictionary dict
2	<u>dict.copy()</u> Returns a shallow copy of dictionary dict
3	<u>dict.fromkeys()</u> Create a new dictionary with keys from seq and values set to value.
4	<u>dict.get(key, default=None)</u> For <i>key</i> key, returns value or default if key not in dictionary

BUILT-IN DICTIONARY FUNCTIONS AND METHODS

Sr.No	Method & Description
5	<u>dict.items()</u> Returns a list of <i>dict</i> 's (key, value) tuple pairs
6	<u>dict.keys()</u> Returns list of dictionary dict's keys

BUILT-IN DICTIONARY FUNCTIONS AND METHODS

Sr.No	Method & Description
7	<u>dict.setdefault(key, default = None)</u> Similar to <code>get()</code> , but will set <code>dict[key] = default</code> if <i>key</i> is not already in <i>dict</i>
8	<u>dict.update(dict2)</u> Adds dictionary <i>dict2</i> 's key-values pairs to <i>dict</i>
9	<u>dict.values()</u> Returns list of dictionary <i>dict</i> 's values

LEN() METHOD

- Python dictionary method len() gives the total length of the dictionary. This would be equal to the number of items in the dictionary.

- Syntax

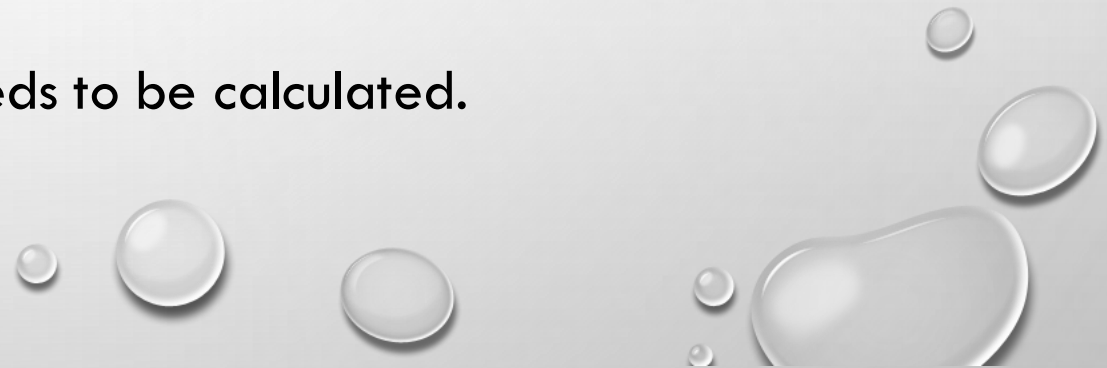
len(dict)

- Parameters

- dict – This is the dictionary, whose length needs to be calculated.

- Return Value

- This method returns the length.



LEN() METHOD

- Example

```
dict1 = {'Name': 'Zara', 'Age': 7};
```

```
Print("Length : ", len (dict1))
```

- Output

Length : 2



STR()

- Python dictionary method `str()` produces a printable string representation of a dictionary.
- Syntax
- `str(dict)`
- Parameters
 - `dict` – This is the dictionary
- Return Value
 - This method returns string representation.



STR()

- Example

```
dict = {'Name': 'Zara', 'Age': 7}
```

```
print ("Equivalent String : ", str (dict))
```

- Output

Equivalent String : {'Name': 'Zara', 'Age': 7}

TYPE()

- Python dictionary method `type()` returns the type of the passed variable. If passed variable is dictionary then it would return a dictionary type.
- Syntax
 - `type(dict)`
- Parameters
 - `dict` – This is the dictionary.
- Return Value
 - This method returns the type of the passed variable.



TYPE()

- Example

```
dict = {'Name': 'Zara', 'Age': 7};
```

```
print ("Variable Type : ", type (dict))
```

- Output

Variable Type : <type 'dict'>



CLEAR()

- The clear() method removes all the elements from a dictionary.
- Syntax
- dictionary.clear()
- Example

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
  
print(car)  
  
car.clear()  
  
print(car)
```

Output:

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}  
{}  

```

COPY()

- The copy() method returns a copy of the specified dictionary
- Syntax
 - dictionary.copy()
- Example

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = car.copy()  
print(x)
```

`{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}`

COPY()

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
x = car.copy()
```

```
print(x)
```

```
x["brand"]="Maruti"
```

```
print(x)
```

```
print(car)
```

{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

{'brand': 'Maruti', 'model': 'Mustang', 'year': 1964}

{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

FROMKEYS()

- The fromkeys() method returns a dictionary with the specified keys and the specified value.
- Syntax
 - `dict.fromkeys(keys, value)`
- Parameter Values
- Parameter Description
 - `keys` Required. An iterable specifying the keys of the new dictionary
 - `value` Optional. The value for all keys. Default value is None

FROMKEYS()

```
x = ('key1', 'key2', 'key3')
```

```
y = 0
```

```
thisdict = dict.fromkeys(x, y)
```

```
print(thisdict)
```

```
{'key1': 0, 'key2': 0, 'key3': 0}
```

```
x = ('key1', 'key2', 'key3')
```

```
thisdict = dict.fromkeys(x)
```

```
print(thisdict)
```

```
{'key1': 0, 'key2': 0, 'key3': 0}
```

```
{'key1': None, 'key2': None, 'key3': None}
```

GET()

- The `get()` method returns the value of the item with the specified key.
- Syntax

`dictionary.get(keyname, value)`

- Parameter Values
- Parameter Description
 - `keyname` Required. The keyname of the item you want to return the value from
 - `value` Optional. A value to return if the specified key does not exist.
 - Default value None

GET()

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
  
x = car.get("model")  
print(x)
```


GET()

- Example
- Try to return the value of an item that do not exist:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = car.get("price", 15000)  
print(x)
```



ITEMS()

- Description
 - The method `items()` returns a list of dict's (key, value) tuple pairs
- Syntax
- `dict.items()`
- Parameters
 - NA
- Return Value
 - This method returns a list of tuple pairs.



ITEMS()

- Example

```
dt = {'Name': 'Zara', 'Age': 7}
```

```
print ("Value : %s" % dt.items())
```

- Result

```
Value : [('Age', 7), ('Name', 'Zara')]
```

KEYS()

- The keys() method returns a view object. The view object contains the keys of the dictionary, as a list.
- The view object will reflect any changes done to the dictionary
- Syntax

`dictionary.keys()`

- Parameter Values
 - No parameters



KEYS()

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
x = car.keys()
```

```
print(x)
```

```
print(type(x))
```

```
dict_keys(['brand', 'model', 'year'])
```

```
<class 'dict_keys'>
```

- Example

- When an item is added in the dictionary, the view object also gets updated:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
x = car.keys()  
car["color"] = "white"  
print(x)
```

```
dict_keys(['brand', 'model', 'year',  
'color'])
```